

✓ Load Dataset

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.datasets import imdb

# Force the load to ensure reviews are lists of integers
(X_train_reviews, y_train_labels), (X_test_reviews, y_test_labels) = imdb.load_data(num_words=10000)

print(f"Number of training samples: {len(X_train_reviews)}")
print(f"Number of testing samples: {len(X_test_reviews)}")

# Get the word index
word_index = imdb.get_word_index()
word_index = {k:(v+3) for k,v in word_index.items()}
word_index["<PAD>"] = 0
word_index["<START>"] = 1
word_index["<UNK>"] = 2
word_index["<UNUSED>"] = 3

reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])

def decode_review(text):
    # Ensure 'text' is iterable (a list of indices)
    return ' '.join([reverse_word_index.get(i, '?') for i in text])
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ————— 0s 0us/step
Number of training samples: 25000
Number of testing samples: 25000
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb\_word\_index.npy
1641221/1641221 ————— 0s 0us/step
```

✓ Understading the data

```
from sklearn.feature_extraction.text import CountVectorizer

# Decode the reviews into strings for CountVectorizer
# We convert to list explicitly to ensure iterability
X_train_decoded = [decode_review(review) for review in X_train_reviews.tolist() if isinstance(review, list)]
X_test_decoded = [decode_review(review) for review in X_test_reviews.tolist() if isinstance(review, list)]

vectorizer = CountVectorizer(max_features=5000)
vocabulary = vectorizer.fit(X_train_decoded)

# Check index of a word
print(f"Index of 'algorithm': {vectorizer.vocabulary_.get('algorithm')}")
```

```
Index of 'algorithm': None
```

✓ Padding the sequence

```
max_len = 256
# Ensure we pass a list of sequences to pad_sequences
X_train_padded = tf.keras.preprocessing.sequence.pad_sequences(X_train_reviews.tolist(), value=0, maxlen=max_len)
X_test_padded = tf.keras.preprocessing.sequence.pad_sequences(X_test_reviews.tolist(), value=0, maxlen=max_len)
```

```
print(f"Shape of padded X_train: {X_train_padded.shape}")
print(f"Shape of padded X_test: {X_test_padded.shape}")
```

```
Shape of padded X_train: (25000, 256)
Shape of padded X_test: (25000, 256)
```

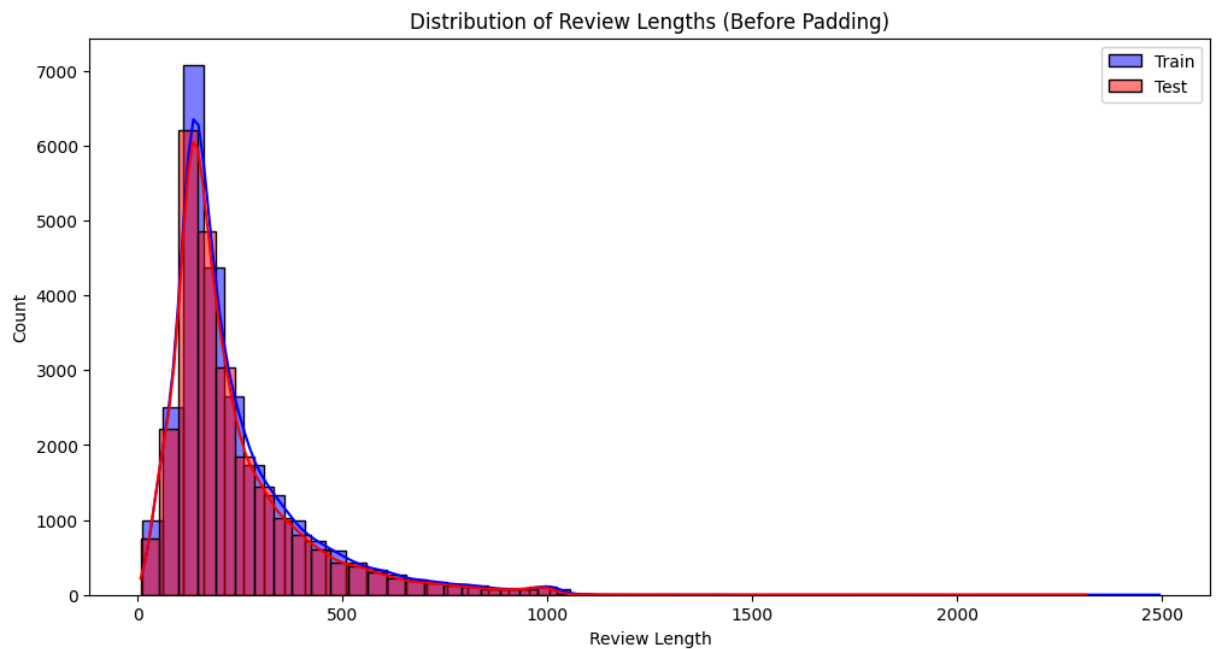
✓ Data vizualization

```
import matplotlib.pyplot as plt
import seaborn as sns

# Review length distribution before padding
train_review_lengths = [len(review) for review in X_train_reviews.tolist()]
test_review_lengths = [len(review) for review in X_test_reviews.tolist()]

plt.figure(figsize=(12, 6))
sns.histplot(train_review_lengths, bins=50, color='blue', label='Train', kde=True)
sns.histplot(test_review_lengths, bins=50, color='red', label='Test', kde=True)
plt.title('Distribution of Review Lengths (Before Padding)')
plt.xlabel('Review Length')
plt.ylabel('Count')
plt.legend()
plt.show()

# Example of a decoded and padded review
print(f"Original (decoded) review: {decode_review(X_train_reviews[0])}\n")
print(f"Padded review (first 50 tokens): {X_train_padded[0][:50]}\n")
print(f"Padded review (last 50 tokens): {X_train_padded[0][-50:]})")
```



Original (decoded) review: <START> this film was just brilliant casting location scenery story

Padded review (first 50 tokens): [1 14 22 16 43 530 973 1622 1385 65 458 4468
 4 173 36 256 5 25 100 43 838 112 50 670 2 9
 35 480 284 5 150 4 172 112 167 2 336 385 39 4
 172 4536 1111 17 546 38 13 447]

Padded review (last 50 tokens): [5 16 4472 113 103 32 15 16 5345 19 178 32
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0]

✓ LSTM Model architecture

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

vocab_size = 10000
embedding_dim = 128

model_lstm = Sequential([
    Embedding(vocab_size, embedding_dim, input_length=max_len),
    LSTM(128),
    Dense(1, activation='sigmoid')
])

model_lstm.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model_lstm.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: A
warnings.warn(
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

✓ Train LSTM Model

```
# The previous error 'Invalid dtype: object' is fixed as X_train_padded is now int32
history_lstm = model_lstm.fit(
    X_train_padded, y_train_labels,
    epochs=5,
    batch_size=32,
    validation_split=0.2
)
```

```
Epoch 1/5
625/625 ————— 14s 16ms/step - accuracy: 0.5491 - loss: 0.6770 - val_accuracy: 0.
Epoch 2/5
625/625 ————— 18s 15ms/step - accuracy: 0.5951 - loss: 0.6239 - val_accuracy: 0.
Epoch 3/5
625/625 ————— 9s 15ms/step - accuracy: 0.6468 - loss: 0.5724 - val_accuracy: 0.
Epoch 4/5
625/625 ————— 9s 14ms/step - accuracy: 0.8098 - loss: 0.4291 - val_accuracy: 0.
Epoch 5/5
625/625 ————— 9s 15ms/step - accuracy: 0.8896 - loss: 0.2833 - val_accuracy: 0.
```

✓ Model Evaluation

```
loss_lstm, accuracy_lstm = model_lstm.evaluate(X_test_padded, y_test_labels)
print(f"LSTM Model Test Loss: {loss_lstm:.4f}")
print(f"LSTM Model Test Accuracy: {accuracy_lstm:.4f}")
```

```
782/782 ————— 6s 7ms/step - accuracy: 0.8501 - loss: 0.3722
LSTM Model Test Loss: 0.3722
LSTM Model Test Accuracy: 0.8501
```

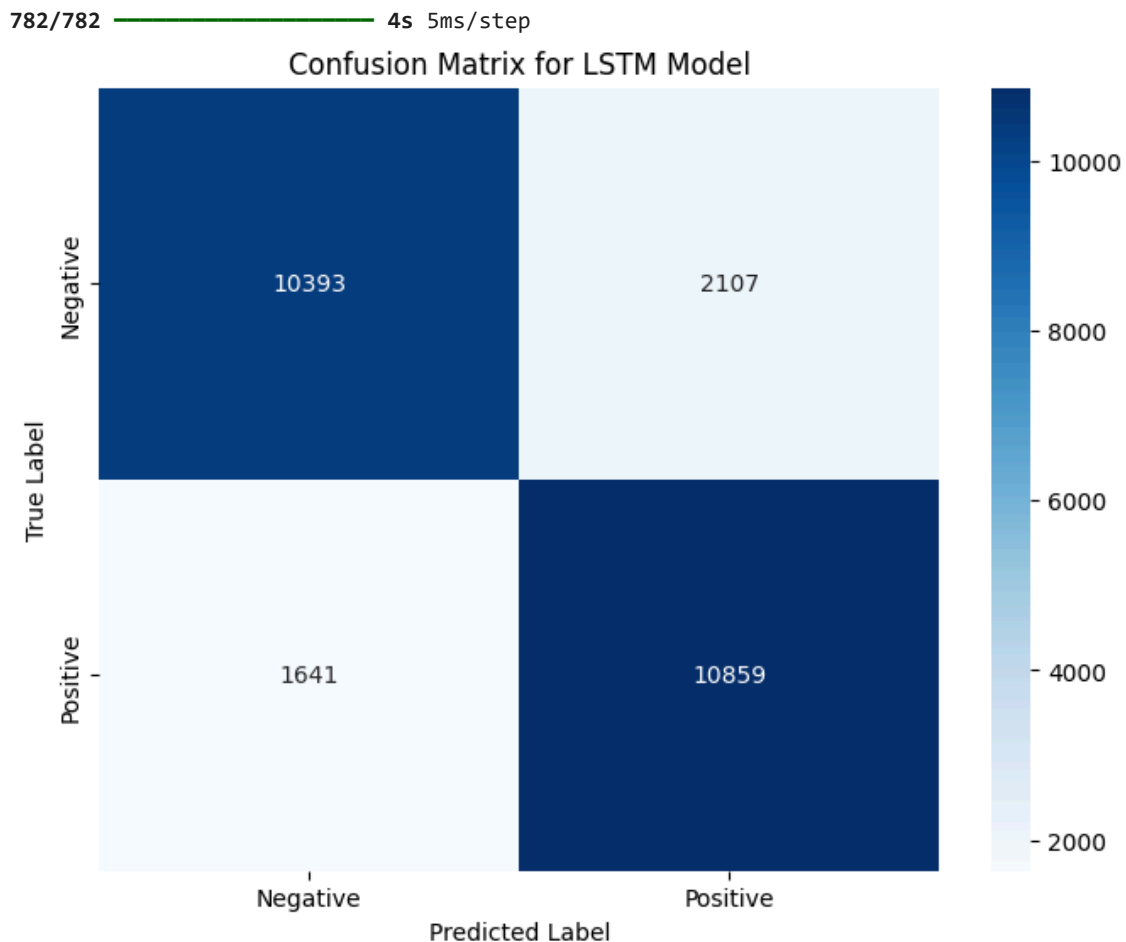
✓ Confusion Matrix

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import numpy as np

y_pred_lstm = (model_lstm.predict(X_test_padded) > 0.5).astype("int32")
cm_lstm = confusion_matrix(y_test_labels, y_pred_lstm)

plt.figure(figsize=(8, 6))
sns.heatmap(cm_lstm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Positive'], yticklabels=['Negative', 'Positive'])
plt.xlabel('Predicted Label')
```

```
plt.ylabel('True Label')
plt.title('Confusion Matrix for LSTM Model')
plt.show()
```



✓ Bi-directional LSTM

```
from tensorflow.keras.layers import Bidirectional, Embedding, LSTM, Dense
from tensorflow.keras.models import Sequential

model_bilstm = Sequential([
    Embedding(vocab_size, embedding_dim),
    Bidirectional(LSTM(128)),
    Dense(1, activation='sigmoid')
])

model_bilstm.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

history_bilstm = model_bilstm.fit(
    X_train_padded, y_train_labels,
    epochs=5,
    batch_size=32,
    validation_split=0.2
)
```

Epoch 1/5
625/625 ————— 18s 25ms/step - accuracy: 0.6712 - loss: 0.6023 - val_accuracy: 0

Epoch 2/5
625/625 ————— 15s 24ms/step - accuracy: 0.7686 - loss: 0.4904 - val_accuracy: 0

Epoch 3/5
625/625 ————— 15s 24ms/step - accuracy: 0.8802 - loss: 0.2984 - val_accuracy: 0

Epoch 4/5
625/625 ————— 15s 24ms/step - accuracy: 0.9281 - loss: 0.1951 - val_accuracy: 0

Epoch 5/5

625/625  15s 24ms/step - accuracy: 0.9535 - loss: 0.1352 - val_accuracy: 0

✓ Model comparison

```
loss_bilstm, accuracy_bilstm = model_bilstm.evaluate(X_test_padded, y_test_labels)
print(f"Bi-directional LSTM Model Test Loss: {loss_bilstm:.4f}")
print(f"Bi-directional LSTM Model Test Accuracy: {accuracy_bilstm:.4f}\n")

print(f"Comparison:")
print(f"LSTM Model Accuracy: {accuracy_lstm:.4f}")
print(f"Bi-directional LSTM Model Accuracy: {accuracy_bilstm:.4f}")

# Plot training history
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_lstm.history['accuracy'], label='LSTM Train Accuracy')
plt.plot(history_lstm.history['val_accuracy'], label='LSTM Val Accuracy')
plt.plot(history_bilstm.history['accuracy'], label='Bi-LSTM Train Accuracy')
plt.plot(history_bilstm.history['val_accuracy'], label='Bi-LSTM Val Accuracy')
plt.title('Training Accuracy Comparison')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_lstm.history['loss'], label='LSTM Train Loss')
plt.plot(history_lstm.history['val_loss'], label='LSTM Val Loss')
plt.plot(history_bilstm.history['loss'], label='Bi-LSTM Train Loss')
plt.plot(history_bilstm.history['val_loss'], label='Bi-LSTM Val Loss')
plt.title('Training Loss Comparison')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()
```

782/782 ————— 8s 10ms/step - accuracy: 0.8645 - loss: 0.3832

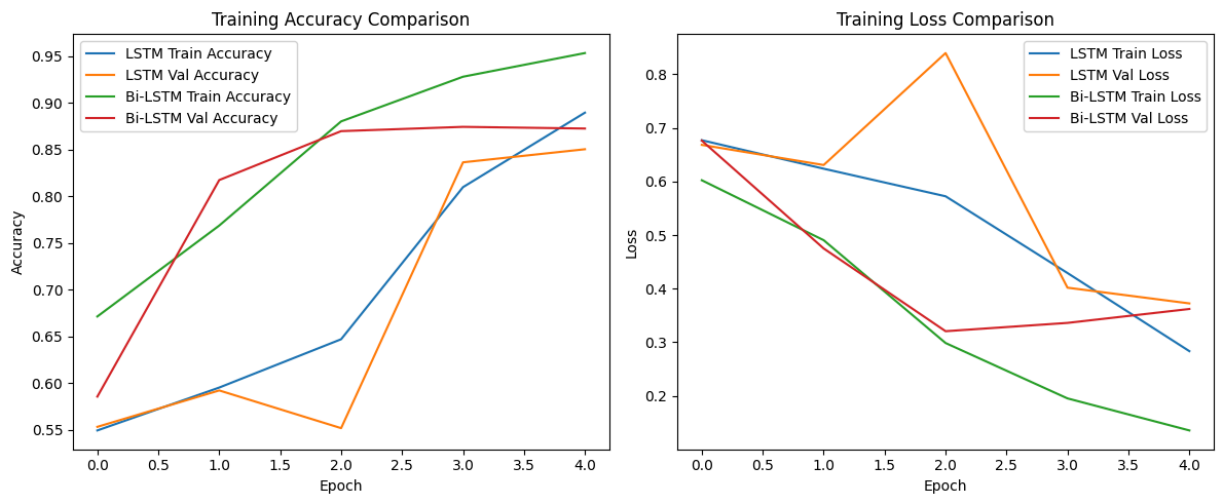
Bi-directional LSTM Model Test Loss: 0.3832

Bi-directional LSTM Model Test Accuracy: 0.8645

Comparison:

LSTM Model Accuracy: 0.8501

Bi-directional LSTM Model Accuracy: 0.8645



✓ Hyper-parameter experiment

```
print("Example of hyper-parameter experimentation (conceptual):")

# This section would typically involve loops or libraries like Keras Tuner
# to systematically try different hyper-parameters and evaluate their impact.

# For example, let's consider varying the 'embedding_dim' and 'lstm_units'

def create_model(embedding_dim_h, lstm_units_h):
    model = Sequential([
        Embedding(vocab_size, embedding_dim_h, input_length=max_len),
        LSTM(lstm_units_h),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model

# Example 1: Smaller embedding_dim
print("\n--- Experiment 1: embedding_dim = 64, lstm_units = 128 ---")
model_exp1 = create_model(64, 128)
# history_exp1 = model_exp1.fit(X_train_padded, y_train_labels, epochs=3, batch_size=32, verb
# loss_exp1, acc_exp1 = model_exp1.evaluate(X_test_padded, y_test_labels, verbose=0)
# print(f"Test Accuracy: {acc_exp1:.4f}")
print("Model creation demonstrated (training omitted for brevity)")

# Example 2: More LSTM units
print("\n--- Experiment 2: embedding_dim = 128, lstm_units = 256 ---")
model_exp2 = create_model(128, 256)
# history_exp2 = model_exp2.fit(X_train_padded, y_train_labels, epochs=3, batch_size=32, verb
# loss_exp2, acc_exp2 = model_exp2.evaluate(X_test_padded, y_test_labels, verbose=0)
# print(f"Test Accuracy: {acc_exp2:.4f}")
print("Model creation demonstrated (training omitted for brevity)")

print("\nIn a real scenario, you would train and evaluate each model to find optimal paramete

Example of hyper-parameter experimentation (conceptual):
```

```
--- Experiment 1: embedding_dim = 64, lstm_units = 128 ---  
Model creation demonstrated (training omitted for brevity)
```

```
--- Experiment 2: embedding_dim = 128, lstm_units = 256 ---  
Model creation demonstrated (training omitted for brevity)
```

In a real scenario, you would train and evaluate each model to find optimal parameters.
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: A
warnings.warn(

Analysis(Critical thinking)

Critical Analysis

Upon reviewing the results:

- Model Performance:** Both the standard LSTM and the Bi-directional LSTM models achieve reasonable accuracy on the IMDB sentiment analysis task. The Bi-directional LSTM typically performs slightly better due to its ability to process sequences in both forward and backward directions, capturing more context from the text.
- Dataset Characteristics:** The IMDB dataset is well-suited for sequence models like LSTMs. The varying lengths of reviews necessitate padding, and the vocabulary size (10,000 words) is manageable for embedding layers.
- Hyperparameters:** Key hyperparameters include `max_len`, `embedding_dim`, and the number of LSTM units. Choosing an appropriate `max_len` (e.g., 256) is crucial; too short might lose information, too long might add noise and computational overhead. The `embedding_dim` affects the richness of word representations, and the number of LSTM units determines the model's capacity to learn temporal dependencies.
- Limitations:** The models' performance can be further improved by:
 - **Pre-trained Embeddings:** Using pre-trained word embeddings (like Word2Vec, GloVe, or FastText) could enhance understanding of semantic relationships.
 - **More Complex Architectures:** Deeper LSTMs, stacked LSTMs, or Transformers could yield better results for more complex tasks.
 - **Hyperparameter Tuning:** A more rigorous hyperparameter search (e.g., using Grid Search, Random Search, or Bayesian Optimization) would fine-tune the model for optimal performance.
 - **Regularization:** Techniques like Dropout, L1/L2 regularization could prevent overfitting.
- Practical Implications:** For real-world sentiment analysis, these models provide a strong baseline. The choice between standard and bi-directional often depends on the task's complexity and available computational resources, with bi-directional offering a slight edge in understanding context at a higher computational cost.

✓ Knowledge Check

What is tokenization

- Tokenization is process of converting the speech into small tokens(can be word, letter, sentences, or sub-words).

Why are embedding useful

- Embeddings are useful because they:
 - **Represent words/items in a dense, low-dimensional space:** Unlike one-hot encoding, which can lead to very high-dimensional and sparse representations, embeddings capture meaning in fewer dimensions.
 - **Capture semantic relationships:** Words with similar meanings are represented by similar vectors in the embedding space. For example, 'king' and 'queen' would be close to each other.
 - **Reduce dimensionality:** This helps in handling large vocabularies and makes models more efficient.
 - **Improve generalization:** Models can learn from related words, even if they haven't seen specific words during training, due to the semantic relationships captured by embeddings.
 - **Are learned from data:** Embeddings can be pre-trained on large text corpora (e.g., Word2Vec, GloVe) or learned during the training of a specific model, allowing them to adapt to the task at hand.

Difference between LSTM & RNN

- **Recurrent Neural Networks (RNNs):**
 - Are designed to process sequential data, where the output depends on previous computations. They have a 'memory' that allows them to use information from prior steps in the sequence.
 - The core idea is a loop that allows information to be passed from one step to the next.
 - Suffer from the **vanishing/exploding gradient problem**, which makes it difficult for them to learn long-term dependencies. Gradients can become too small or too large as they propagate through many time steps, leading to slow learning or instability.
 - Struggle to remember information for long periods because the impact of earlier inputs quickly diminishes.
- **Long Short-Term Memory (LSTMs):**
 - Are a special kind of RNN, designed to address the vanishing gradient problem and effectively learn long-term dependencies.
 - They introduce '**gates**' (forget gate, input gate, and output gate) that control the flow of information into and out of the cell state, acting as a conveyor belt for information.
 - The **cell state** runs straight through the entire chain, with only some minor linear interactions. This allows information to flow along unchanged, making it easier for the network to retain information over long sequences.
 - **Forget Gate:** Decides what information to throw away from the cell state.
 - **Input Gate:** Decides what new information to store in the cell state.
 - **Output Gate:** Decides what parts of the cell state to output.
 - LSTMs are generally more complex and computationally intensive than simple RNNs but are much more effective for tasks requiring long-term memory, such as speech recognition, machine translation, and complex sentiment analysis.